



PRACTICA 4_AUTOMATIZACIÓN DE TAREAS

RECURSOS

- Tener una máquina (real o virtual) con GNU/Linux.

OBJETIVOS

- Manejar comandos automatizar tareas linux

INTRODUCCIÓN

1. VENTAJAS DE LA AUTOMATIZACIÓN DE TAREAS

La automatización se ha convertido en una herramienta indispensable en el mundo de la tecnología, permitiendo a las organizaciones y a los profesionales de TI simplificar tareas repetitivas, reducir errores y aumentar la eficiencia en sus operaciones.

- La automatización permite realizar tareas en cuestión de segundos que podrían llevar horas si se hicieran manualmente. Esto no solo ahorra tiempo, sino que también reduce la probabilidad de errores humanos.
- Las tareas automatizadas se ejecutan con una lógica predefinida y constante, garantizando la uniformidad en la configuración y en los procedimientos

Para programar tareas en GNU/Linux disponemos de dos herramientas: **crontab** - **at**

Diferencias:

- **crontab**: Ejecuciones periódicas de uno o varios procesos.
- **at**: Una única ejecución de uno o varios procesos.
- **crontab**: No mantiene el "entorno" en la ejecución.
- **at**: Mantiene las variables de entorno y ruta de ejecución actual, así como cualquier otra variable definida en ese momento.
- **crontab**: Es el más usado y conocido. Hay mucha documentación sobre él.
- **at**: Es un desconocido, pero que aporta grandes posibilidades.
- **crontab**: Su servicio implicado en su ejecución es el cron.



- **at**: El servicio que facilita su ejecución es el anacron.

2. EJECUTAR TAREAS A UNA DETERMINADA HORA:AT

at

- Ejecuta comandos en la fecha y hora especificados una única vez. Además permite enviar un email como recordatorio de la tarea.
- Permite ver el contenido de la tarea que se vaya a ejecutar.

atq

- Muestra la cola/lista de trabajos existentes en la lista de tareas pendientes.
- Comando alias de at -l

atrm

- Permite eliminar una determinada tarea de la lista de tareas pendientes de ejecución.
- Comando alias de at -r

Pasos para crear una tarea única utilizando el comando at

1. Ejecutar el comando at indicando hora y día (opcional) en que se desee se ejecute la tarea.
2. Escribir cada comando a ejecutar en una línea, pulsar INTRO para finalizar la entrada de cada comando.
3. Finalizar la entrada de la tarea at, con CTRL+D

Cosas importantes a tener en cuenta:

1. Los comandos de ejecutan con /bin/sh no con /bin/bash
2. Si la hora de ejecución ya pasó la tarea no se ejecutará.

Sintaxis de at

at hora [fecha]

Ejemplos de uso:

***# Tarea que se ejecutará a las 11:00 del día actual
at 11:00***

***# Tarea que se ejecutará el primero Domingo (Sun) a las 20:00
at 20:00 Sun***

***# Tarea que se ejecutará a las 4:00 el 14/03/2023
at 04:00 2023-03-14***

***# Tarea que se ejecutará a las 4:00 el 14/03/2023
at 4am 22.10.14***



Se permite usar at indicando un 'incremento' de tiempo:

*# Ejecutar tarea dentro de 3 horas
at now + 3 hours*

*# Ejecutar mañana (incremento especial = + 1 days)
at 15:00 tomorrow*

RESUMEN DEL COMANDO AT Y ACCIONES

Crear Tarea

- **at hh:mm YYYY-MM-DD**

Usar hora y fecha. Si se usa sólo la hora se ejecutará en el día actual.

Escribir cada comando + INTRO

Finalizar creacion de tarea con CTRL+D

Listar Tareas / Ver contenido de la tarea

- **atq / at -l**

Se puede usar el comando at con el parámetro -l (listar)

- **at -c Num**

Ver los comandos que ejecutará la tarea Num

Borrar Tareas

- **atrm Num / at -r Num**

Se puede usar también at con el parámetro -r (remove/borrar)

Ruta de las Tareas

Todas las tareas se almacenan en el directorio: **/var/spool/cron/atjobs**

3. EJECUTAR TAREAS PERIÓDICAMENTE: CRON/CRONTAB

Cron es un demonio (que es como se conoce a un proceso en segundo plano) que se ejecuta desde el mismo instante en el que arranca el sistema operativo. **Cron** se encargará de comprobar si existe alguna tarea (**job**) para ser ejecutada, de acuerdo a la hora configurada en el propio sistema operativo.

El administrador **Cron** ejecuta comandos de forma automática y en segundo plano, es decir, sin alterar la operatividad y funciones de Linux. Los procesos se ejecutarán sin que el usuario tenga que hacer ninguna acción e incluso sin notarlo.

El usuario puede configurar tareas infinitas con fechas y horas de ejecución personalizadas para activarse de forma horaria, diaria, semanal, mensual o anual

Crontab es esencialmente una lista donde los usuarios agregan sus tareas y trabajos para automatizarlos, y esto tiene serie opciones que podrían simplificar dichas tareas aún más.



Crontab es un fichero que contiene los comandos que serán ejecutados por el administrador Cron forma personalizada por el usuario. Los comandos almacenados en el **Crontab** son archivos de texto con los códigos de programación de cada tarea a ejecutar.

Del mismo modo, las tareas de **Crontab** contiene la hora y fecha de ejecución así como los comandos de repetición.

La principal diferencia entre ambos es que Cron es un administrador de tareas y Crontab es un fichero de comandos.

Como usuario podrás agregar tareas personalizadas utilizando una serie de comandos. Veamos cómo hacerlo:

- **m**: Se refiere al minuto en que se va a ejecutar el comando con valores de 0 a 59.
- **h**: Indica la hora exacta en formato de 24 horas (0 a 23).
- **dom**: Es una acrónimo de day of month (día del mes). En formatos de 1 a 30, se puede especificar qué día del mes queremos que se ejecute la tarea.
- **dow**: "day of week" o día de la semana específico en un formato de 7 días el día de la semana. También se puede usar las 3 primeras letras del día en inglés (mon, tue, wed, thu, fri, sat, sun).
- **user**: Este comando especifica quién será el usuario que ejecuta el comando.
- **command**: Nombra al comando o ruta del script que se va a ejecutar.

Ejemplos de tareas de Cron y Crontab

1. Veamos un ejemplo con una tarea repetida en unos meses específicos:
*** * * jul,sep,nov * /script/job.sh**
2. Supongamos que deseamos que el sistema haga un backup cada viernes a las 10:00 de la mañana:
00 10 * * fri /scripts/job.sh
3. Si deseamos que el sistema haga un respaldo el 10 de abril a las 7:00 a.m. podríamos crear el script:
00 7 10 4 * /home/script/backup.sh

Comando crontab

- Para remplazar el archivo existente por otro que defina el usuario se debe utilizar el siguiente comando:
crontab archivo
- Para editar el archivo existente en la actualidad se utiliza el comando que ya hemos visto a lo largo de este artículo:
crontab -e
- Listar todas las tareas existentes en el crontab del usuario:



crontab -l

- Borrar el crontab que está configurado:

crontab -d

- Definir el directorio en el que se almacenará el archivo de crontab. Para realizar esta operación se deben tener permisos de ejecución en dicho directorio:

crontab -c dir

- Instrucción para manejar el crontab de otros usuarios existentes en el sistema:

crontab -u usuario

- Este comando te permite borrar el archivo crontab de manera permanente:

crontab -r

SECUENCIA/DESARROLLO

1 Ejemplo1: Sigue los pasos para ejecutar una tarea con at

1.1 Primer paso : Instalar at

```
root@debian64:~# apt install at
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
at ya está en su versión más reciente (3.2.5-1+b1).
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
root@debian64:~#
```

1.2 Segundo paso: Iniciar el servicio

```
root@debian64:~# systemctl start atd_
```

```
root@debian64:~# systemctl enable --now atd
Synchronizing state of atd.service with SysV service script with /lib
Executing: /lib/systemd/systemd-sysv-install enable atd
root@debian64:~# _
```

1.3 Tercer paso: Programar la tarea

```
Archivo Máquina Ver Entrada Dispositivos Ayuda
user@debian64:~$ echo `date` >> "$HOME/fecha_at.txt" | at now + 2 minutes
```

1.4 Otra forma:

```
user@debian64:~$ at now + 3 minutes
warning: commands will be executed using /bin/sh
at Thu Nov 13 22:05:00 2025
at> date >>"$HOME/fecha_at.txt"
at> <EOT>
job 8 at Thu Nov 13 22:05:00 2025
user@debian64:~$ atq
8          Thu Nov 13 22:05:00 2025 a user
user@debian64:~$ _
```

2 Ejercicio 1: Programar tareas puntuales

Para poder recibir un correo utilizamos mail → instalamos el paquete mailutils → sólo local; opciones por defecto.

- 2.1 Comprueba que el demonio atd este ejecutándose.
- 2.2 Programa la ejecución del comando "ls /" dentro de 3 minutos. Comprueba que at enviara un e-mail con el resultado de la instrucción ejecutada.
- 2.3 Lista las tareas pendientes que hay en la cola.
- 2.4 Programa la orden "ls /tmp > \$HOME/salida_tmp" dentro de 3 minutos. Ten en cuenta que, al haber redirigido la salida de la instrucción a ejecutar, at no avisara sobre la realización de la tarea.
- 2.5 Averigua /Comprueba que hay en la cola de tareas.
- 2.6 Para finalizar, comprueba que las dos ordenes programadas se han ejecutado. La primera habrá mandado el resultado por e-mail, mientras que la segunda habrá creado el fichero con la salida de la orden. (Para leer el correo puedes usar la orden mail).

3 Ejercicio 3: Programar tareas

- 3.1 Programa una tarea que cree en tu HOME el fichero **usr_conectados** que se ejecutará de lunes a viernes a las 3 de la tarde y contendrá la información de usuario – terminal – hora de todos los usuarios que estén conectados a esa hora.
- 3.2 Programa una tarea para ejecutarse a las 12:00 que elimine los ficheros que terminen por .pru del HOME.
 - 3.2.1 Visualiza la lista de tareas programadas y comprueba que está la que acabas de programar.
 - 3.2.2 Borra esta tarea programada.
- 3.3 Crea como root una tarea que actualice el sistema. Se ejecutará todos los viernes a las 8 de la tarde.
 - 3.3.1 Lista tareas cron de root. Visualiza el fichero que contiene estas tareas.



- 3.4 Entra al sistema con el usuario "profe" y programa la herramienta Cron para que en el día de hoy y de este mes, durante la hora actual y cada 5 minutos se ejecute la orden `/bin/ls -R $HOME`, y el resultado se guarde en el fichero `/tmp/tareasaso_hora:minutos`. Comprueba si esta haciendo la tarea observando el contenido `/tmp//tareasaso_hora:minutos`.
- 3.5 Una vez que has visto que se ejecuta la tarea, elimina la entrada introducida por el usuario profe en el **Cron**.